Now provide me entire frontend and backend codes in python and uv.I have downloaded vlm - qwen3-vl-2b-instruct and llm - qwen2.5-0.5b through foundry local.Refer below code.I need UI for the below code only where user will submit weforms url and business logic and after submitting,the test result will appear.

```
client = OpenAI(
base_url="http://127.0.0.1:46127/v1",
api_key="token-if-required",
)

class TestCase(BaseModel):
scenario_name: str = Field(description="Short description of what we are testing.")
input_value: str = Field(description="The exact data to type into the form field.")
expected_behavior: str = Field(
description="What the website should do, e.g. 'Show error' or 'Allow login'."
)

class TestSuite(BaseModel):
test_cases: List[TestCase]

business_rule = (
"Coupons only apply if the shopping cart total is over ₹1,000. "
"Under ₹1,000, the coupon must be rejected."
)

form_field_context = (
"The field is a text box named 'txtCartTotal' that accepts numeric values."
)

prompt = f"""
You are a QA Automation Engineer.

Generate test scenarios for the form field below, including:

- normal/valid values
- boundary values
- invalid values
```

- empty input
- special characters
- very large values

Return ONLY valid JSON. Do not include explanations, Markdown, or code fences.

Use exactly this JSON structure:
{{

"test_cases":
$$"scenario_name" : "string", "input_value" : "string", "expected_behavior" : "string"$$

}}

Business Rule: {business_rule}

Target Form Field: {form_field_context}
"""

response = client.chat.completions.create(
model="qwen2.5-0.5b",

messages=
$$"role" : "system", "content" : ("You are a precise QA assistant. """ Always respond with a single va$$
,

temperature=0.2,
)

raw_content = response.choices[0].message.content
print("Raw model output:")
print(raw_content)

try:
data = json.loads(raw_content)
generated_suite = TestSuite.model_validate(data)

print("\n Generated Test Suite for Automation:")
print("-" * 60)

for index, case in enumerate(generated_suite.test_cases, 1):
print(f"Test #{index}: {case.scenario_name}")
print(f" Input to Type: {case.input_value}")
print(f" Expected Result: {case.expected_behavior}")
print("-" * 60)

except (json.JSONDecodeError, ValidationError) as exc:
print("\n The model did not return valid schema-conforming JSON.")
print(exc)

The project structure is
[Webform URL + Your Business Rules]
|
▼

1. LLM (Brain) ——► Generates smart test data (e.g., Age: 17, Age: 0, Age: 200)
   |
   ▼
2. Playwright (Hands) ——► Automatically opens the browser, types the data, and clicks "Submit"
   |
   ▼
3. VLM / LLM (Eyes & Judgment) ——► Looks at the result page and decides if it "Passed" or "Failed"

Below is a complete **Python + uv** project. It gives users a web UI to enter a **WeForms URL** and **business logic**, generates test cases with `qwen2.5-0.5b`, runs them in a browser with Playwright, then asks `qwen3-vl-2b-instruct` to judge the result screenshot as **PASS** or **FAIL**. Playwright supports browser automation and screenshots through its Python API. [1] [2] [3]

Project structure

```
weforms-tester/
├── pyproject.toml
├── .env.example
├── README.md
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── schemas.py
│   ├── llm.py
│   └── runner.py
└── static/
    └── index.html
```

1. Create the project

```
uv init weforms-tester
cd weforms-tester
uv add fastapi "uvicorn[standard]" pydantic pydantic-settings python-dotenv openai pl
uv run playwright install chromium
```

Playwright's Python library can launch Chromium, open pages, interact with elements, and take screenshots. [2] [3] [4] [1]

2. `pyproject.toml`

```
[project]
name = "weforms-tester"
version = "0.1.0"
description = "AI-powered WeForms UI test generator and runner"
requires-python = ">=3.10"
dependencies = [
    "fastapi>=0.115.0",
    "uvicorn[standard]>=0.30.0",
    "pydantic>=2.7.0",
    "pydantic-settings>=2.3.0",
    "python-dotenv>=1.0.1",
    "openai>=1.30.0",
    "playwright>=1.45.0",
]

[tool.uv]
dev-dependencies = []
```

3. `.env.example`

Create a file named `.env` from this example:

```
FOUNDRY_BASE_URL=http://127.0.0.1:46127/v1
FOUNDRY_API_KEY=token-if-required
LLM_MODEL=qwen2.5-0.5b
VLM_MODEL=qwen3-vl-2b-instruct
MAX_TEST_CASES=8
```

Do **not** commit the real `.env` file. Keep the Foundry Local endpoint and API key on the backend only.

4. `app/__init__.py`

Create an empty file:

```

```

5. `app/schemas.py`

```
from typing import List, Literal
from pydantic import BaseModel, Field


class TestCase(BaseModel):
    scenario_name: str = Field(
        description="Short description of what is being tested."
    )
    input_value: str = Field(
        description="Exact value to type into the first visible text input."
```

```
    )
    expected_behavior: str = Field(
        description=(
            "Expected visible result after submission, for example: "
            "'Shows coupon applied' or 'Shows error message'."
        )
    )


class TestSuite(BaseModel):
    test_cases: List[TestCase] = Field(default_factory=list)


class TestRequest(BaseModel):
    url: str = Field(
        min_length=1,
        max_length=2048,
        description="WeForms form URL to test.",
    )
    business_logic: str = Field(
        min_length=5,
        max_length=4000,
        description="Business rule to test, written in plain English.",
    )


class TestResult(BaseModel):
    scenario_name: str
    input_value: str
    expected_behavior: str
    status: Literal["PASS", "FAIL", "ERROR"]
    actual_behavior: str
    screenshot_path: str | None = None
    error: str | None = None


class TestRunResponse(BaseModel):
    url: str
    business_logic: str
    results: List[TestResult]
    passed: int
    failed: int
    errors: int
```

6. `app/llm.py`

This module generates test cases and judges each screenshot using your local Foundry models.

```
import base64
import json
import re
from pathlib import Path

from openai import OpenAI
from pydantic import ValidationError
```

```
from app.schemas import TestSuite

client = OpenAI(
    base_url="http://127.0.0.1:46127/v1",
    api_key="token-if-required",
)

LLM_MODEL = "qwen2.5-0.5b"
VLM_MODEL = "qwen3-vl-2b-instruct"
MAX_TEST_CASES = 8


def build_generation_prompt(business_logic: str) -> str:
    return f"""
You are a QA Automation Engineer.

Generate test scenarios for a web form based on the business rule below.

Business Rule:
{business_logic}

Target form:
The form contains a visible text input field. The tester will enter the
provided value into the first visible text input and submit the form.

Generate between 4 and {MAX_TEST_CASES} test cases, including:
- normal/valid values
- boundary values
- invalid values
- empty input
- special characters
- very large values

Return ONLY valid JSON. Do not include explanations, Markdown, or code fences.

Use exactly this JSON structure:
{{
  "test_cases": [
    {{
      "scenario_name": "string",
      "input_value": "string",
      "expected_behavior": "string"
    }}
  ]
}}

Important:
- Keep every input_value below 50 characters.
- Do not repeat the same input_value.
- expected_behavior must describe a visible page result.
"""


def generate_test_cases(business_logic: str) -> TestSuite:
    response = client.chat.completions.create(
```

```
        model=LLM_MODEL,
        messages=[
            {
                "role": "system",
                "content": (
                    "You are a precise QA assistant. "
                    "Always respond with one valid JSON object. "
                    "Never include Markdown, explanations, or code fences."
                ),
            },
            {
                "role": "user",
                "content": build_generation_prompt(business_logic),
            },
        ],
        temperature=0.2,
        max_tokens=1200,
    )

    raw = response.choices[^0].message.content or ""
    raw = extract_json(raw)

    try:
        data = json.loads(raw)
        return TestSuite.model_validate(data)
    except (json.JSONDecodeError, ValidationError) as exc:
        raise ValueError(
            "The LLM did not return valid schema-conforming JSON."
        ) from exc


def extract_json(text: str) -> str:
    text = text.strip()

    if text.startswith("```"):
        text = re.sub(r"^```(?:json)?\s*", "", text)
        text = re.sub(r"\s*```$", "", text)

    start = text.find("{")
    end = text.rfind("}")

    if start == -1 or end == -1 or end < start:
        raise ValueError("No JSON object found in LLM response.")

    return text[start : end + 1]


def encode_image(image_path: str) -> str:
    with Path(image_path).open("rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")


def judge_result(
    expected_behavior: str,
    screenshot_path: str,
) -> tuple[str, str]:
```

```
    image_data = encode_image(screenshot_path)

    response = client.chat.completions.create(
        model=VLM_MODEL,
        messages=[
            {
                "role": "system",
                "content": (
                    "You are a QA judge. Compare the screenshot with the "
                    "expected behavior. Reply with only valid JSON."
                ),
            },
            {
                "role": "user",
                "content": [
                    {
                        "type": "text",
                        "text": (
                            "Expected behavior:\n"
                            f"{expected_behavior}\n\n"
                            "Decide whether the screenshot satisfies the "
                            "expected behavior.\n\n"
                            "Return only this JSON:\n"
                            '{"status": "PASS" or "FAIL", '
                            '"actual_behavior": "short description"}'
                        ),
                    },
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"{image_data}"
                        },
                    },
                ],
            },
        ],
        temperature=0.0,
        max_tokens=300,
    )

    raw = response.choices.message.content or ""[^0]
    raw = extract_json(raw)

    try:
        data = json.loads(raw)
        status = str(data.get("status", "FAIL")).upper()
        actual = str(data.get("actual_behavior", "No description provided."))

        if status not in {"PASS", "FAIL"}:
            status = "FAIL"

        return status, actual
    except (json.JSONDecodeError, ValidationError):
        return "FAIL", "The vision model returned an invalid judgment."
```

7. `app/runner.py`

This module opens the WeForms URL, enters each generated value into the **first visible text input**, submits the form, takes a screenshot, and asks the VLM to judge it.

```
import re
from datetime import datetime
from pathlib import Path
from urllib.parse import urlparse

from playwright.sync_api import sync_playwright

from app.llm import judge_result
from app.schemas import TestCase, TestResult


def is_safe_url(url: str) -> bool:
    try:
        parsed = urlparse(url)
        return parsed.scheme in {"http", "https"} and bool(parsed.netloc)
    except Exception:
        return False


def run_test_suite(url: str, test_cases: list[TestCase]) -> list[TestResult]:
    if not is_safe_url(url):
        raise ValueError("Enter a valid HTTP or HTTPS URL.")

    run_id = datetime.now().strftime("%Y%m%d_%H%M%S")
    screenshot_dir = Path("screenshots") / run_id
    screenshot_dir.mkdir(parents=True, exist_ok=True)

    results: list[TestResult] = []

    with sync_playwright() as playwright:
        browser = playwright.chromium.launch(headless=True)
        page = browser.new_page()

        try:
            page.goto(url, wait_until="domcontentloaded", timeout=30000)
        except Exception as exc:
            browser.close()
            raise ValueError(f"Could not open the URL: {exc}") from exc

        for index, test_case in enumerate(test_cases, start=1):
            result = execute_case(
                page=page,
                url=url,
                test_case=test_case,
                screenshot_dir=screenshot_dir,
                index=index,
            )
            results.append(result)

        browser.close()
```

```
    return results


def execute_case(
    page,
    url: str,
    test_case: TestCase,
    screenshot_dir: Path,
    index: int,
) -> TestResult:
    screenshot_path = screenshot_dir / f"test_{index}.png"
    error_message = None

    try:
        page.goto(url, wait_until="domcontentloaded", timeout=30000)

        text_input = page.locator(
            "input[type='text']:visible, "
            "input[type='number']:visible, "
            "input[type='email']:visible, "
            "input:not([type]):visible"
        ).first

        text_input.wait_for(state="visible", timeout=10000)
        text_input.fill(test_case.input_value)

        submit_button = page.locator(
            "button[type='submit']:visible, "
            "input[type='submit']:visible, "
            "button:has-text('Submit'):visible"
        ).first

        if submit_button.count() > 0:
            submit_button.click()
        else:
            page.keyboard.press("Enter")

        page.wait_for_timeout(3000)
        page.screenshot(path=str(screenshot_path), full_page=True)

        status, actual_behavior = judge_result(
            expected_behavior=test_case.expected_behavior,
            screenshot_path=str(screenshot_path),
        )

        return TestResult(
            scenario_name=test_case.scenario_name,
            input_value=test_case.input_value,
            expected_behavior=test_case.expected_behavior,
            status=status,
            actual_behavior=actual_behavior,
            screenshot_path=str(screenshot_path),
        )

    except Exception as exc:
```

```
        error_message = str(exc)

        try:
            page.screenshot(path=str(screenshot_path), full_page=True)
        except Exception:
            screenshot_path = None

        return TestResult(
            scenario_name=test_case.scenario_name,
            input_value=test_case.input_value,
            expected_behavior=test_case.expected_behavior,
            status="ERROR",
            actual_behavior="Test could not be completed.",
            screenshot_path=str(screenshot_path) if screenshot_path else None,
            error=error_message,
        )
```

The runner uses Playwright's `page.goto()`, locator, `fill()`, `click()`, and `screenshot()` methods. Playwright's `Page` API supports navigation, element interaction, and screenshots. [3] [1]

8. `app/main.py`

This is the FastAPI backend. It serves the UI and exposes two endpoints: one to generate test cases and one to run them.

```
from pathlib import Path

from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles

from app.llm import generate_test_cases
from app.runner import run_test_suite
from app.schemas import TestRequest, TestRunResponse, TestSuite

app = FastAPI(title="WeForms AI Tester")

BASE_DIR = Path(__file__).resolve().parent.parent
STATIC_DIR = BASE_DIR / "static"

app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")


@app.get("/")
def home():
    return FileResponse(STATIC_DIR / "index.html")


@app.post("/api/generate")
def generate_tests(request: TestRequest) -> TestSuite:
    try:
        return generate_test_cases(request.business_logic)
    except ValueError as exc:
        raise HTTPException(status_code=422, detail=str(exc)) from exc
```

```
    except Exception as exc:
        raise HTTPException(
            status_code=502,
            detail=f"LLM generation failed: {exc}",
        ) from exc


@app.post("/api/run")
def run_tests(request: TestRequest) -> TestRunResponse:
    try:
        suite = generate_test_cases(request.business_logic)
        results = run_test_suite(request.url, suite.test_cases)
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    except Exception as exc:
        raise HTTPException(
            status_code=500,
            detail=f"Test execution failed: {exc}",
        ) from exc

    passed = sum(result.status == "PASS" for result in results)
    failed = sum(result.status == "FAIL" for result in results)
    errors = sum(result.status == "ERROR" for result in results)

    return TestRunResponse(
        url=request.url,
        business_logic=request.business_logic,
        results=results,
        passed=passed,
        failed=failed,
        errors=errors,
    )
```

9. `static/index.html`

This is the complete frontend UI. Save it in `static/index.html`.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>WeForms AI Tester</title>

  <style>
    * {
      box-sizing: border-box;
      font-family: Arial, sans-serif;
    }

    body {
      margin: 0;
      background: #f4f6f8;
      color: #1f2933;
    }
```

```
.container {
  max-width: 1100px;
  margin: 0 auto;
  padding: 30px 20px 60px;
}

header {
  margin-bottom: 25px;
}

h1 {
  margin: 0 0 8px;
  font-size: 28px;
}

p {
  margin: 0;
  color: #52606d;
}

.card {
  background: white;
  border-radius: 12px;
  padding: 24px;
  margin-bottom: 24px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.07);
}

label {
  display: block;
  font-weight: bold;
  margin-bottom: 7px;
}

input,
textarea {
  width: 100%;
  padding: 12px;
  border: 1px solid #cbd2d9;
  border-radius: 8px;
  font-size: 15px;
  margin-bottom: 18px;
}

textarea {
  min-height: 130px;
  resize: vertical;
}

button {
  background: #2563eb;
  color: white;
  border: none;
  padding: 12px 22px;
  border-radius: 8px;
```

```
      font-size: 15px;
      cursor: pointer;
    }

    button:disabled {
      background: #9fb3c8;
      cursor: not-allowed;
    }

    .secondary {
      background: #52606d;
      margin-left: 10px;
    }

    .hidden {
      display: none;
    }

    .status {
      margin: 15px 0;
      font-weight: bold;
    }

    .summary {
      display: flex;
      gap: 15px;
      margin-bottom: 20px;
      flex-wrap: wrap;
    }

    .summary div {
      background: #f0f4f8;
      padding: 14px 20px;
      border-radius: 8px;
      font-weight: bold;
    }

    table {
      width: 100%;
      border-collapse: collapse;
      background: white;
    }

    th,
    td {
      border: 1px solid #e4e7eb;
      padding: 10px;
      text-align: left;
      vertical-align: top;
    }

    th {
      background: #f0f4f8;
    }

    .PASS {
```

```
      color: #047857;
      font-weight: bold;
    }

    .FAIL {
      color: #b91c1c;
      font-weight: bold;
    }

    .ERROR {
      color: #b45309;
      font-weight: bold;
    }

    .error-box {
      background: #fee2e2;
      color: #991b1b;
      padding: 14px;
      border-radius: 8px;
      margin-bottom: 20px;
    }

    .screenshot-link {
      color: #2563eb;
      text-decoration: none;
    }
  </style>
</head>

<body>
  <div class="container">
    <header>
      <h1>WeForms AI Tester</h1>
      <p>
        Enter a WeForms URL and business rule. The app will generate test cases,
        fill and submit the form, and judge the result.
      </p>
    </header>

    <div id="errorBox" class="error-box hidden"></div>

    <section class="card">
      <label for="url">WeForms URL</label>
      <input
        id="url"
        type="url"
        placeholder="https://your-site.com/weforms-form/"
      />

      <label for="businessLogic">Business Logic</label>
      <textarea
        id="businessLogic"
        placeholder="Example: Coupons only apply if the shopping cart total is over
      ></textarea>

      <button id="runButton">Generate and Run Tests</button>
```

```
      <button id="clearButton" class="secondary">Clear</button>

      <div id="status" class="status hidden"></div>
    </section>

    <section id="resultsSection" class="hidden">
      <div class="card">
        <h2>Test Results</h2>

        <div class="summary">
          <div>Total: <span id="totalCount">0</span></div>
          <div>Passed: <span id="passedCount">0</span></div>
          <div>Failed: <span id="failedCount">0</span></div>
          <div>Errors: <span id="errorCount">0</span></div>
        </div>

        <table>
          <thead>
            <tr>
              <th>#</th>
              <th>Scenario</th>
              <th>Input</th>
              <th>Expected</th>
              <th>Actual</th>
              <th>Status</th>
              <th>Screenshot</th>
            </tr>
          </thead>
          <tbody id="resultsBody"></tbody>
        </table>
      </div>
    </section>
  </div>

  <script>
    const urlInput = document.getElementById("url");
    const businessLogicInput = document.getElementById("businessLogic");
    const runButton = document.getElementById("runButton");
    const clearButton = document.getElementById("clearButton");
    const statusDiv = document.getElementById("status");
    const errorBox = document.getElementById("errorBox");
    const resultsSection = document.getElementById("resultsSection");
    const resultsBody = document.getElementById("resultsBody");

    function showError(message) {
      errorBox.textContent = message;
      errorBox.classList.remove("hidden");
    }

    function hideError() {
      errorBox.classList.add("hidden");
    }

    function setStatus(message) {
      statusDiv.textContent = message;
      statusDiv.classList.remove("hidden");
```

```
}

function clearResults() {
  resultsSection.classList.add("hidden");
  resultsBody.innerHTML = "";
  hideError();
  statusDiv.classList.add("hidden");
}

clearButton.addEventListener("click", () => {
  urlInput.value = "";
  businessLogicInput.value = "";
  clearResults();
});

runButton.addEventListener("click", async () => {
  const url = urlInput.value.trim();
  const businessLogic = businessLogicInput.value.trim();

  hideError();
  clearResults();

  if (!url || !businessLogic) {
    showError("Please enter both the WeForms URL and business logic.");
    return;
  }

  runButton.disabled = true;
  setStatus("Generating test cases and running browser tests...");

  try {
    const response = await fetch("/api/run", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        url,
        business_logic: businessLogic
      })
    });

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.detail || "Test run failed.");
    }

    renderResults(data);
    setStatus("Test run completed.");
  } catch (error) {
    showError(error.message);
    setStatus("");
  } finally {
    runButton.disabled = false;
  }
```

```
    });

    function renderResults(data) {
      document.getElementById("totalCount").textContent = data.results.length;
      document.getElementById("passedCount").textContent = data.passed;
      document.getElementById("failedCount").textContent = data.failed;
      document.getElementById("errorCount").textContent = data.errors;

      resultsBody.innerHTML = "";

      data.results.forEach((result, index) => {
        const row = document.createElement("tr");

        const screenshot = result.screenshot_path
          ? `<a class="screenshot-link" href="/${result.screenshot_path}" target="_bl
          : "—";

        row.innerHTML = `
          <td>${index + 1}</td>
          <td>${escapeHtml(result.scenario_name)}</td>
          <td>${escapeHtml(result.input_value)}</td>
          <td>${escapeHtml(result.expected_behavior)}</td>
          <td>${escapeHtml(result.actual_behavior)}</td>
          <td class="${result.status}">${result.status}</td>
          <td>${screenshot}</td>
        `;

        resultsBody.appendChild(row);
      });

      resultsSection.classList.remove("hidden");
    }

    function escapeHtml(value) {
      const div = document.createElement("div");
      div.textContent = value ?? "";
      return div.innerHTML;
    }
  </script>
</body>
</html>
```

10. README.md

```
# WeForms AI Tester

This app generates and runs UI test cases for a WeForms form.

## Requirements

- Python 3.10+
- uv
- Foundry Local running at:
  http://127.0.0.1:46127/v1
- Models downloaded:
```

```
  - qwen2.5-0.5b
  - qwen3-vl-2b-instruct
- Chromium browser for Playwright

## Setup

```bash
uv sync
uv run playwright install chromium
cp .env.example .env
```
```

Run the app

```
uv run uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Open:

```
http://127.0.0.1:8000
```

How to use

1. Enter the WeForms form URL.
2. Enter the business rule.
3. Click **Generate and Run Tests**.
4. Review the generated test cases and PASS/FAIL results.
5. Open a screenshot to inspect what the browser saw.

Example business logic

```
Coupons only apply if the shopping cart total is over ₹1,000.
Under ₹1,000, the coupon must be rejected.
```

Notes

- The app fills the first visible text input on the page.
- It clicks the first visible submit button, or presses Enter if no button is found.
- The VLM reviews the post-submission screenshot and returns PASS or FAIL.
- Screenshots are stored in the `screenshots/` folder.

```
## Run it

```bash
uv sync
```
```

```
uv run playwright install chromium
uv run uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Then open:

```
http://127.0.0.1:8000
```

Important limitation

This version fills the **first visible text input** and clicks the **first visible submit button**. That works for a simple WeForms form with one main input, but it will not reliably handle multi-field forms, dropdowns, checkboxes, CAPTCHAs, or complex layouts.

For a production-quality version, the LLM should also return a **CSS selector** or field label for each test case, and the backend should validate that selector before allowing Playwright to use it. Do not let the LLM directly execute browser actions or arbitrary JavaScript; validate and constrain its output first. [5] [6] [7]

⁂

1. https://playwright.dev/python/docs/api/class-playwright
2. https://playwright.dev/python/docs/library
3. https://playwright.dev/python/docs/api/class-page
4. https://playwright.dev/python/docs/intro
5. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
6. https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html
7. https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet
8. https://playwright.dev/python/docs/network
9. https://playwright.dev/python/docs/emulation
10. https://playwright.dev/python/docs/api/class-apiresponse
11. https://playwright.dev/python/docs/api/class-browser
12. https://playwright.dev/python/docs/api/class-browsercontext
13. https://playwright.dev/python/docs/codegen
14. https://playwright.dev/python/docs/pom
15. https://playwright.dev/python/docs/api/class-browsertype
16. https://github.com/microsoft/playwright-python
17. https://playwright.dev/python/docs/api-testing
18. https://playwright.dev/python/docs/api/class-apirequestcontext